Hussam Almaweri
CIS-375
Project Proposal & SOW
Project Proposal: Syndex

Project Information

Project Title: Syndex

Team Member: Hussam Almaweri (Individual Project)

Description: Syndex is a web application that combines Canvas assignments and manually entered tasks into one dashboard. Students can track progress, set priorities, and generate study schedules based on deadlines and availability.

Intended Users and Environment: University students managing coursework across multiple classes. The application will run locally through a web browser.

Technologies: Python, Flask, HTML/CSS/JavaScript, SQLite, Python icalendar and zoneinfo libraries, Git, and VS Code.

Problem Statement and Objectives

Canvas displays assignment deadlines but does not automatically organize students' available study time around their workload. Students must independently determine which assignments to prioritize, estimate how long tasks will take, and plan when to complete them.

Syndex addresses this problem by combining assignment management, progress tracking, prioritization, and study schedule generation.

Objectives:

- Import deadlines from Canvas using an ICS calendar feed.
- Allow users to create and manage academic tasks.
- Track assignment completion, priorities, and estimated work durations.
- Generate suggested study schedules based on deadlines and availability.

Expected Outcome: A functional web application that helps students organize and plan their academic workload.

Hussam Almaweri
CIS-375
Project Proposal & SOW

Project Scope and Statement of Work

Core Features:

- **Canvas Integration:** Import assignments and deadlines through an ICS feed with on-demand synchronization.
- **Task Management:** Create, edit, delete, and organize academic tasks.
- **Progress Tracking:** Track task statuses and calculate completion percentages.
- **Priority Management:** Assign priority levels and estimated completion times.
- **Study Scheduler:** Generate study sessions based on deadlines, task priorities, and available time.
- **Dashboard:** Display assignments, priorities, progress, and generated schedules.

Optional Features: Automatic Canvas synchronization, course color coding, and automatic rescheduling.

Limitations: Canvas integration will be read-only. The ICS feed excludes Canvas To Do items and is subject to feed limits. Study schedules will use a rule-based algorithm, not artificial intelligence. Public hosting is outside the project scope.

Deliverables: Working web application, SQLite database, source code, technical documentation, testing results, and final demonstration.

Initial Software Requirements

Functional Requirements:

- FR-01: Import and synchronize assignments from a Canvas ICS URL.
- FR-02: Create, edit, and delete manual tasks.
- FR-03: Track task completion status and calculate progress.
- FR-04: Assign priorities and estimated work durations.
- FR-05: Record available study periods.
- FR-06: Generate suggested study sessions based on deadlines and availability, warning when insufficient time exists.
- FR-07: Display tasks and generated schedules through a unified dashboard.

- FR-08: Prevent duplicate imports and preserve locally recorded task information during synchronization.

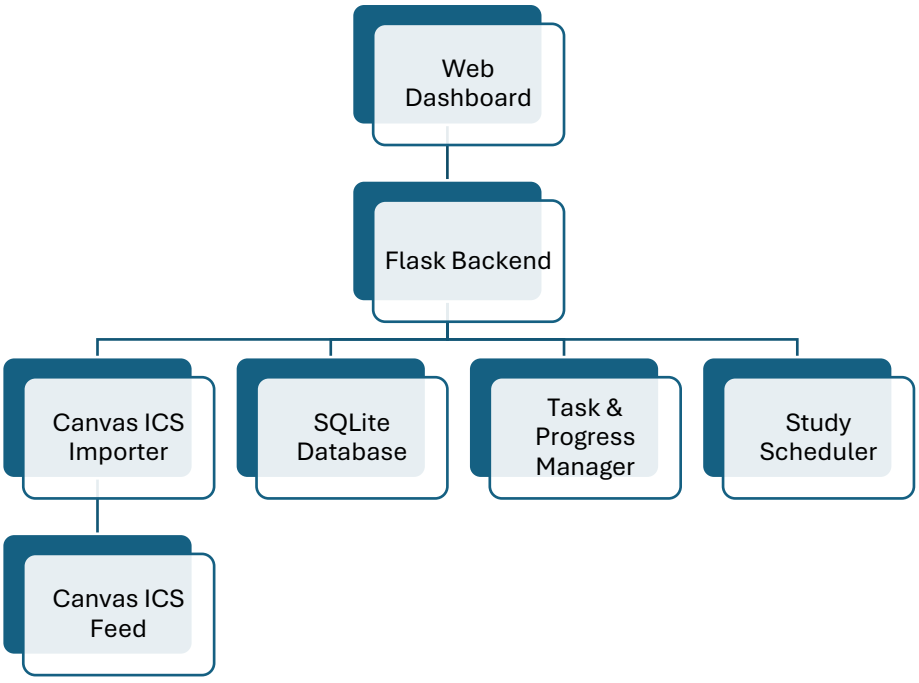
Nonfunctional Requirements:

- **Usability:** Provide a responsive and easy-to-navigate interface.
- **Reliability:** Maintain access to stored tasks if Canvas synchronization fails.
- **Security:** Protect the Canvas ICS URL from unauthorized access.
- **Data Integrity:** Handle dates and timezones correctly and preserve stored task information.

User Interactions: Users will manage tasks, update progress, set priorities, enter study availability, generate schedules, and synchronize Canvas through the web dashboard.

System Interfaces: Flask will retrieve Canvas ICS data over HTTPS, interact with SQLite for storage, and execute the study scheduling algorithm.

System Block Diagram:



The backend coordinates Canvas importing, task management, and schedule generation while storing application data in SQLite.

Hussam Almaweri
CIS-375
Project Proposal & SOW

Software Design and Development Approach

Syndex will follow an MVC-style architecture using Flask for application logic, SQLite for data storage, and HTML/CSS/JavaScript for the user interface.

The main components will include an ICS importer, task manager, progress tracker, and rule-based study scheduler. Existing Python libraries will support calendar parsing and timezone handling.

Development will follow Agile practices through short iterations. Each iteration will include implementation, testing, and progress reviews, with changes documented as needed.

Since Flask and web development are relatively new to me, the initial development phase will include learning Flask fundamentals, such as routing, HTML templates, form handling, and database integration. These concepts will first be practiced through a basic task management application before integrating Canvas and the study scheduling features.

Agile Project Planning and Schedule

Activity	Planned Work	Deliverable
Planning	Requirements and scope	Project proposal
Iteration 1 (Nov 2-14)	Learn Flask fundamentals, set up the database, and implement basic task CRUD.	Basic task tracker
Iteration 2 (Nov 15-28)	ICS importing, synchronization, status tracking	Canvas-integrated tracker
Iteration 3 (Nov 29-Dec 5)	Priorities, progress, study scheduler	Functional planner
Iteration 4 (Dec 6-12)	Integration testing and defect correction	Tested application
Final Delivery	Documentation and demonstration	Final report and software

The schedule may be adjusted to meet the final project deadline. Core features will take priority over optional improvements.

Team Responsibilities and Collaboration

As an individual project, I will be responsible for planning, requirements analysis, design, implementation, integration, testing, documentation, technical reviews, and the final presentation. Development progress, completed tasks, and design decisions will be documented throughout the project.

Hussam Almaweri
CIS-375
Project Proposal & SOW

Testing and Validation Plan

Unit Testing: Verify ICS parsing, task management functions, progress calculations, and scheduling logic.

Integration Testing: Confirm that Canvas assignments and manual tasks work together without duplicates or lost completion statuses.

Schedule Testing: Verify that generated sessions respect deadlines and availability and identify insufficient study time.

Failure Testing: Test invalid ICS links, network errors, and incomplete task information.

Acceptance Criteria: Users must be able to import assignments, manage tasks, track progress, assign priorities, and generate a feasible study schedule or receive an appropriate warning. Defects will be documented and corrected throughout development.

Project Risks and Challenges

- **Canvas Integration:** Inconsistent calendar data or synchronization failures. *Mitigation:* Validate imported data and retain previously stored assignments.
- **Scheduling Complexity:** Estimated work may exceed available time. *Mitigation:* Use a simplified rule-based algorithm and display warnings.
- **Component Integration:** Combining multiple features may cause database inconsistencies. *Mitigation:* Perform integration testing throughout development.
- **Learning New Technologies:** Limited experience with Flask and web development may slow down implementation. *Mitigation:* Use documentation, tutorials, and small practice implementations to learn the necessary concepts before integrating more complex features.
- **Time Constraints:** Individual development may limit available time. *Mitigation:* Prioritize core features and defer optional enhancements.